

Administrator Guide

Project: Enterprise Quantum Computing Platform — QFT Bank **Student:** Syed Muhammad Tahoor Ali **Diploma:** EduQual Level 6, Diploma in Artificial Intelligence Operations (ANPP-OP)

1. Purpose

This guide is for whoever operates the platform after it is deployed: monitoring its health, understanding its persistence behaviour, reading its metrics, and troubleshooting the issues most likely to arise. It assumes the platform is running via the Kubernetes path described in the Deployment Guide.

2. Health and Status

The single most useful operational endpoint is `/health`:

```
curl http://<minikube-ip>:30080/health
```

It returns `{"status":"healthy","storage_backend":"..."}`. The `storage_backend` value is the fastest way to know whether the platform connected to PostgreSQL (`postgresql`) or is running on the in-memory fallback (`in-memory`). If you expect a database and see `in-memory`, that is the first thing to investigate (Section 6).

Pod-level status:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

3. Persistence Model

Execution history is stored in a **single unified executions table** with a `module` discriminator column and a JSON result payload, rather than four near-identical per-module tables. Every history-writing module — quantum circuits, algorithms, portfolio optimization, and PQC — routes through one persistence layer, so behaviour is consistent platform-wide.

The defining property is **graceful degradation**. At startup the platform tests the database connection. If PostgreSQL is reachable, records are written there and survive restarts. If it is not reachable, the platform logs a warning and writes to an in-memory dictionary instead of crashing. Even at runtime, if an individual database write fails, that record falls back to memory rather than being lost.

Operationally this means:

- In a healthy Kubernetes or Docker Compose deployment, history persists across restarts in PostgreSQL.
- If the backend starts before PostgreSQL is ready (pods start in parallel), it comes up on the in-memory fallback. A `kubectl rollout restart deployment/backend` after the database is ready reconnects it.
- During a live demonstration, a database hiccup degrades gracefully rather than taking the platform down.

The active backend is always reported by `/health`, so an administrator never has to guess which store is in use.

4. Monitoring and Observability

The backend is instrumented with `prometheus-fastapi-instrumentator`, which exposes a `/metrics` endpoint in Prometheus format. The stack:

Component	NodePort	Role
Prometheus	30090	Scrapes the backend's <code>/metrics</code> ; stores time series
Grafana	30300	Queries Prometheus; renders dashboards
Loki	(in-cluster)	Log aggregation backend; wired as a Grafana datasource

Useful queries in Prometheus or Grafana include request rate (`rate(http_requests_total[1m])`) and per-endpoint latency. Grafana is pre-wired with the Prometheus datasource.

Log aggregation status. Loki is deployed and operational and is registered as a second Grafana datasource. Promtail (the log-shipping agent) is deployed with verified RBAC and corrected configuration but is not currently discovering log targets on this Minikube environment; the log-path mounting appears environment-specific. Container logs remain fully available through `kubectl logs` in the meantime. The full diagnostic history is recorded in `SCOPE_DECISIONS.md`.

5. Security Posture

Security is addressed primarily at build time, through the CI/CD pipeline, and reinforced by container hardening at runtime.

Build-time scanning (four layers in the Jenkins pipeline): Bandit for static analysis of the Python source, pip-audit for known CVEs in dependencies, Trivy for vulnerabilities in the built image, and an Open Policy Agent (Rego) policy gate that validates every Kubernetes manifest before images are built. The policy gate is a hard gate, not advisory — a manifest that violates baseline reliability or security rules fails the build.

Runtime hardening. The backend container runs as a non-root user (UID 1000) with `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, and all Linux capabilities dropped. This can be verified on a running pod:

```
POD=$(kubectl get pod -l app=backend -o jsonpath='{.items[0].metadata.name}')
kubectl exec $POD -- id
```

The output shows `uid=1000(appuser)`, confirming the process is not root.

Network posture. Only the frontend service is exposed externally (NodePort 30080), plus the two monitoring UIs. The backend, PostgreSQL, and Redis are `ClusterIP`-only. The browser's single entry point is nginx, which reverse-proxies API calls to the backend over the in-cluster network.

Identity and access management (Keycloak) was deliberately not deployed; the reasoning and the production alternative are documented in `SCOPE_DECISIONS.md`.

6. Troubleshooting

6.1 `/health` reports in-memory when a database is expected

The backend started before PostgreSQL was ready, or cannot reach it. Confirm PostgreSQL is running (`kubectl get pods`), then restart the backend to reconnect:

```
kubectl rollout restart deployment/backend
kubectl rollout status deployment/backend
```

Re-check `/health`; it should now report `postgresql`.

6.2 502 Bad Gateway on `/api/*` through the frontend

This indicates nginx cannot reach or resolve the backend. Two causes have been seen and fixed in this project, both worth knowing:

- Name resolution.** nginx must proxy to the fully-qualified service name `backend.default.svc.cluster.local`, not the short name `backend`. nginx's resolver does not apply the pod's DNS search domains the way the C library does, so the short name can fail to resolve even when `kubectl exec ... wget backend:8000` succeeds.
- Proxy path handling.** When `proxy_pass` uses a variable (required for request-time DNS resolution), nginx does not substitute the location prefix. The upstream must be built as `proxy_pass $backend$request_uri`; so the original path is forwarded intact; otherwise the backend receives a wrong path and returns 404.

Both are already fixed in the committed `frontend/nginx.conf`. If a 502 recurs, first confirm the backend pod is Ready and reachable in-cluster:

```
FPOD=$(kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}')
kubectl exec $FPOD -- wget -qO- http://backend.default.svc.cluster.local:8000/health
```

6.3 A pod will not start after an image change

Because the Kubernetes manifests use `imagePullPolicy: Never`, Kubernetes uses the image already inside Minikube's Docker daemon. If a rebuild was not loaded into that daemon, the pod uses the old image. Ensure `eval $(minikube docker-env)` was run before `docker build`, then restart the deployment.

6.4 The backend crash-loops after a securityContext change

If `runAsNonRoot: true` is asserted but the image has no non-root user, the pod fails to start. The provided backend image builds a UID-1000 user specifically to satisfy this. If modifying the image, keep the `USER appuser` directive.

7. Routine Operations

Task	Command
View all pods	<code>kubectl get pods</code>
Tail backend logs	<code>kubectl logs -f deploy/backend</code>
Restart the backend	<code>kubectl rollout restart deployment/backend</code>
Check active storage backend	<code>curl http://<ip>:30080/health</code>
Open Grafana	<code>http://<minikube-ip>:30300</code>

Open Prometheus	<code>http://<minikube-ip>:30090</code>
Confirm non-root backend	<code>kubectrl exec <backend-pod> -- id</code>

8. Escalation Notes

The platform is a proof-of-concept on a single-node cluster. Production hardening — enforced startup ordering, an identity layer, distributed tracing, a full SIEM, and a multi-node managed cluster — is documented as future work in `SCOPE_DECISIONS.md` and the project README. The graceful-degradation design means that in the interim, transient failures degrade the platform's persistence rather than its availability.